

Q34. Sudoku Solver for Finding All Possible Solutions

1. Aim

To develop a backtracking-based Sudoku solver that finds all possible valid solutions for a partially filled Sudoku grid.

2. Problem Statement

Given a partially filled 9×9 Sudoku grid, fill all empty cells such that every row, column, and 3×3 subgrid contains numbers from 1 to 9 without repetition.

The algorithm should find **all possible valid solutions**.

3. Objectives

- Solve the given Sudoku using backtracking.
- Explore every possible legal value.
- Apply pruning using row, column, and subgrid rules.
- Continue searching after finding one solution.
- Ensure that no valid solution is missed.

4. Input Details

A partially filled 9×9 Sudoku grid is given.

- 0 represents an empty cell.
- Numbers 1–9 represent filled cells.

Example:

```
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
```

0 6 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9

5. Constraints

- Each row must contain numbers 1 to 9 without repetition.
- Each column must contain numbers 1 to 9 without repetition.
- Each 3×3 subgrid must contain numbers 1 to 9 without repetition.
- Given numbers cannot be changed.
- Every empty cell must receive a valid number.

6. Algorithm Used

Backtracking with Pruning

7. Algorithm

1. Find an empty cell in the Sudoku grid.
2. Try numbers from 1 to 9.
3. Check whether the number is valid in the row.
4. Check whether the number is valid in the column.
5. Check whether the number is valid in the 3×3 subgrid.
6. If valid, place the number.
7. Recursively solve the remaining empty cells.
8. If a complete grid is obtained, store it as a solution.
9. Remove the placed number and backtrack.
10. Continue trying other possible numbers.
11. Repeat until all possible solutions are found.

Program:

```
1 def is_valid(grid, row, col, num):
2
3     # Check row
4     for i in range(9):
5         if grid[row][i] == num:
6             return False
7
8     # Check column
9     for i in range(9):
10        if grid[i][col] == num:
11            return False
12
13    # Check 3x3 subgrid
14    start_row = (row // 3) * 3
15    start_col = (col // 3) * 3
16
17    for i in range(start_row, start_row + 3):
18        for j in range(start_col, start_col + 3):
19            if grid[i][j] == num:
20                return False
21
22    return True
23
24
25 def find_empty_cell(grid):
```

OUTPUT:

```
Sudoku Solver
-----
Number of Solutions: 1

Solution 1
[5, 3, 4, 6, 7, 8, 9, 1, 2]
[6, 7, 2, 1, 9, 5, 3, 4, 8]
[1, 9, 8, 3, 4, 2, 5, 6, 7]
[8, 5, 9, 7, 6, 1, 4, 2, 3]
[4, 2, 6, 8, 5, 3, 7, 9, 1]
[7, 1, 3, 9, 2, 4, 8, 5, 6]
[9, 6, 1, 5, 3, 7, 2, 8, 4]
[2, 8, 7, 4, 1, 9, 6, 3, 5]
[3, 4, 5, 2, 8, 6, 1, 7, 9]
```

10. Analysis

The algorithm selects an empty cell and tries values from 1 to 9.

If a value violates the row, column, or 3×3 subgrid rule, it is immediately rejected. This is the **pruning step**.

For a valid value, the algorithm places it and recursively continues. After returning, the value is removed so that other possibilities can be explored.

11. Pruning

Pruning is applied whenever:

- The number already exists in the row.
- The number already exists in the column.
- The number already exists in the 3×3 subgrid.

Invalid choices are rejected immediately, reducing unnecessary searches.

12. Completeness

The algorithm does not stop after finding the first solution.

After storing a solution, it backtracks and continues trying other possible values. Therefore, **all valid Sudoku solutions are explored and stored**.

13. Complexity

If there are N empty cells, the worst-case time complexity is approximately:

$$O(9^N)$$

Space complexity:

$$O(N)$$

This represents the recursion stack, excluding the memory required to store all solutions.

14. Scalability

The algorithm is suitable for standard 9×9 Sudoku puzzles. The search space can become very large for puzzles with many empty cells, but row, column, and subgrid pruning significantly reduces unnecessary searches.

15. Result

The Sudoku puzzle was successfully solved using backtracking.

Number of Solutions = 1

The completed grid satisfies all row, column, and 3×3 subgrid constraints.

16. Conclusion

The Sudoku Solver successfully uses backtracking to find all valid solutions. Pruning eliminates illegal values early, while recursive calls explore all remaining possibilities. Since the algorithm continues after finding a solution, it ensures that **all possible valid Sudoku solutions are identified without missing any solution.**